



ML Based Solution for DSMAC System in Aerial Navigation

Shivam Kashyap¹(✉), Jagadish Shivamurthy², and C. R. S. Kumar¹

¹ Department of Computer Science and Engineering, Defence Institute of Advance Technology, Girinagar, Pune, India

shivam4776kashyap@gmail.com suthikshnkumar@diat.ac.inf

² Aeronautical Development Establishment Lab, Defence Research and Development Organisation, Bangalore, India
s.jagadish.ade@gov.in

Abstract. Modern navigation systems face significant challenges due to the limitations of traditional methods such as Inertial Navigation Systems (INS) and Global Navigation Satellite System (GNSS). INS suffers from cumulative drift errors, while GPS is vulnerable to jamming, spoofing, and signal loss, particularly in adversarial or obstructed environments. Digital Scene Matching and Area Correlation (DSMAC) is one of effective method used to alleviate these issues. Traditionally, DSMAC is implemented using image processing techniques. In this paper, we introduce ML based solutions which are easy and fast to build and trained for a specific mission. A custom dataset of 35,000 aerial images was created based on the planned flight path and altitudes, to train and evaluate various Convolutional Neural Network (CNN)-based models, including Facebook AI Similarity Search (FAISS) and traditional image matching model i.e. Oriented FAST and rotated BRIEF (ORB). These models, originally designed for tasks such as image segmentation and image classification, were fine-tuned specifically for aerial image matching. Through this fine-tuning process and subsequent testing, performance metrics were obtained, demonstrating the potential of these models to handle the unique requirements of aerial navigation such as, handling images captured from various angles, image distortion due to weather conditions and real-time processing with limited compute resources. Additionally, a data creation application was developed around Google Earth Pro, which generates datasets based on marked areas provided in a KML file. This application automates dataset creation by extracting aerial images of specified regions, automating the data preparation process.

Keywords: Image Matching · Convolutional Neural Networks (CNNs) · Aerial Navigation · GPS-Denied Environments · Autonomous Aerial Navigation Systems · DataGeneration

1 Introduction

Aerial image analysis has gained prominence in applications like urban planning, disaster management, and GIS. However, accurate image matching in aerial

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026

R. P. Yadav et al. (Eds.): ICDSA 2025, LNNS 1721, pp. 266–283, 2026.

https://doi.org/10.1007/978-3-032-10753-4_23

datasets remains challenging due to variations in altitude, lighting, and landscape complexity. This research aims to improve aerial image matching using deep learning techniques and efficient search algorithms.

A dataset of 35,000 high-resolution aerial images was created, capturing an urban environment at a fixed 100-meter altitude. Images were organized in a grid format for comprehensive regional coverage. The dataset was generated automatically using Google Earth Pro, with images extracted based on predefined coordinates, serving as the foundation for testing image matching models.

Initially, the Oriented FAST and Rotated BRIEF (ORB) feature-based method proposed by Rublee et al. [10] was used for matching. While ORB showed reasonable accuracy in structured areas, it struggled with images lacking distinct features. To address this, deep learning methods were introduced. A U-Net convolutional neural network (CNN), introduced by Ronneberger et al. [8], was used for feature extraction and matching, leveraging its segmentation strengths, though it fell short in robust image matching.

To improve retrieval efficiency and accuracy, the Facebook AI Similarity Search (FAISS) framework by Johnson et al. [6] with a ResNet-50 backbone from He et al. [4] was implemented. Comparative analysis showed that FAISS outperformed ORB and U-Net in processing time and accuracy, especially for images with complex structures like buildings and roads.

The rest of this paper is organized as follows: Sect. 2 reviews relevant literature and previous work in this area. Section 3 explains the data creation methodology and details how the dataset generation process was automated using the DCAS (Dataset Creation and Stitching) Application. Section 4 presents the experimental results and analysis. Section 5 discusses the challenges faced and limitations of the study. Section 6 outlines potential future work. The paper concludes with Acknowledgments and References.

The key contributions of this research are: (1) Development of an automated dataset generation system for aerial navigation that significantly reduces preparation time; (2) Fine-tuned and comparison of traditional feature-based methods (ORB) with modern deep learning approaches (FAISS with ResNet-50 [4,6]) for image matching; (3) Comprehensive evaluation of these models under various conditions to determine their suitability for DSMAC systems; and (4) A practical framework that can be adapted for real-world aerial navigation in GPS-denied environments.

2 Related Work

Image matching is a critical component in computer vision, supporting various applications like navigation, mapping, and surveillance. The evolution of this field has been driven by advancements in computational capabilities and AI technologies. Early approaches relied on traditional feature-based methods like SIFT and SURF for image alignment. Zhou et al. [15] presented key advancements in this area, while Bai, Y. [2] developed improved matching techniques. While effective, these methods are limited by computational inefficiency and sensitivity to variations in lighting, scale, and perspective.

Recent advancements in artificial intelligence have led to significant improvements in image matching accuracy and robustness. SuperGlue, for instance, employs Graph Neural Networks (GNNs) with self- and cross-attention mechanisms to reason over spatial and visual relationships between features. Sarlin et al. [11] demonstrated that this approach outperforms traditional heuristic-based methods like Scale-Invariant Feature Transform (SIFT) and Random Sample Consensus (RANSAC), especially under wide-baseline and occluded scenarios.

Vision-inertial fusion systems further enhance navigation reliability by integrating visual data with inertial measurements. Zhou et al. [15] demonstrated this through UAV experiments, showing algorithms like phase congruency for feature extraction remain robust against environmental variations such as lighting and seasonal changes. Nassar et al. [7] extended the approach with deep learning, making such systems crucial for ensuring stable navigation in UAVs operating under GPS-denied conditions.

Wang et al. [14] developed deep learning-based frameworks like U-Net and FAISS which have also demonstrated strong capabilities in image feature extraction and matching. Semantic segmentation techniques, such as those used in UAV geolocation systems, provide contextual information that improves geospatial alignment accuracy. These methods form the basis of scalable and adaptable navigation systems for real-world applications.

To better understand how existing methods compare in terms of their objectives and limitations, a summary of key techniques is presented in Table 1. This table outlines various contributions from the literature, including works by Zhou et al. [15] on vision/inertial fusion for UAV navigation, Bai, Y. [2] on image detection algorithms, and Nassar et al. [7] who utilized deep CNNs for UAV geolocalization. Other noteworthy contributions include Irani and Christ [5], who focused on scene matching in the context of Tomahawk missile guidance, and Sarlin et al. [11], who proposed deep homography-based techniques for accurate image alignment. While these approaches offer valuable insights, many still face practical limitations, such as high computational demands, environment-specific applicability, or sensitivity to lighting conditions.

With these existing approaches as context, we now turn to our methodology for creating a comprehensive dataset to train and evaluate our proposed models.

3 Dataset Creation

Navigating aircraft and missiles in GPS-denied environments is challenging due to signal jamming, obstructions in urban or forested areas, and denied-access areas. Image matching techniques, which compare real-time images with pre-existing aerial data, present a promising solution. For strategic missions in diverse regions, generating tailored, high-quality, area-specific datasets is essential. The success of this approach relies heavily on the availability of such datasets.

Table 1. Comparison of Existing Image Matching Approaches for Navigation

Reference	Method	Objective	Limitations
Zhou et al. [15]	Vision/Inertial Fusion	UAV Navigation	Limited to structured environments
Bai, Y. [2]	Image Detection Algorithms	General Image Matching	High computational cost
Nassar et al. [7]	Deep CNN	UAV Geolocalization	Requires extensive training data
Irani and Christ [5]	Scene Matching	Tomahawk Guidance	Works only in specific terrain types
Sarlin et al. [11]	Deep Homography	Precise Image Alignment	Sensitive to lighting conditions

A Windows-based application was developed to automate aerial image dataset generation for training Deep Learning Models. Users upload a Keyhole Markup Language (KML) file defining the target area and set parameters like image quality and ratio of overlap. The increase in overlapping will help the model to find the closest match. The application then automatically creates the dataset, reducing manual effort and ensuring consistent data quality. This tool enhances mission readiness by enabling rapid, precise dataset creation, enabling training of the models for the specific mission.

3.1 Dataset Creation Methodology

Accurate image matching in aerial navigation relies on high-quality datasets that comprehensively capture the geographical features of an area. In this study, a structured approach was adopted to generate a large-scale aerial image dataset that would serve as a foundation for testing various image matching models.

Three different categories of images were collected:

- **Low-Quality Images with Overlapping:** Captured at a resolution of 1920×964 pixels (HD), ensuring redundancy and increased feature availability.
- **Low-Quality Images without Overlapping:** Captured at the same resolution but without image duplication to ensure unique feature points in each frame.
- **High-Quality Images with Overlapping:** Captured at 3840×2160 pixels (4K) to retain finer image details and enhance feature extraction accuracy.

A total of **77 GB** of image data was generated, covering an extensive **30 square kilometers** of urban terrain. The dataset was structured in a grid-based manner to facilitate systematic analysis. The naming convention followed a matrix representation:

- **Overlapping Images:** Arranged in a 22×700 **matrix**, resulting in a total of 15,400 images. And 50% overlap is in between the images.

- **Non-Overlapping Images:** Sequentially numbered from `img1,1` to `img11,400`, yielding a total of **4,400** images.

The total number of images for the overlapping dataset is mathematically expressed as:

$$N_{\text{overlap}} = R \times C = 22 \times 700 = 15,400 \quad (1)$$

where R and C represent the number of rows and columns, respectively. Similarly, for the non-overlapping dataset:

$$N_{\text{non-overlap}} = 4,400 \quad (2)$$

Relation Between Altitude and Ground Area Covered. In **Google Earth Pro**, the ground area visible depends on:

- **Altitude:** Higher altitude increases the visible area.
- **Field of View (FOV):** Typically around **60°** horizontally.
- **Aspect Ratio:** Screen shape (e.g., 16:9) affects horizontal and vertical coverage.

Approximate Calculation. At an altitude of **100 m** with a **60° FOV**, the ground width (W) covered is:

$$W = 2 \times (100 \times \tan(30^\circ)) \approx 115.5 \text{ m}$$

For a 16:9 screen, vertical coverage is about **65 m**.

Ground Area Covered:

$$\text{Area} \approx 115.5 \text{ m} \times 65 \text{ m} = 7,500 \text{ m}^2$$

Table 2. Dataset Summary

Dataset(images)	No. of Images	Size of Dataset	Size of Image	Creation Time
Low Q* with O*	15,400	15.4 GB	1920 × 964	8 h
Low Q* without O*	4,400	1.77 GB	1920 × 964	4 hr
High Q* with O*	15,400	68.5 GB	3840 × 2160	30 hr

* Q means Quality

* O means Overlapping

Table 2 provides a comprehensive summary of the datasets generated through the DCAS application under varying conditions of image quality and overlap. The low-quality datasets, both overlapping and non-overlapping, consist

of images with a resolution of 1920×964 pixels, created in relatively shorter durations (8 h and 4 h, respectively). In contrast, the high-quality overlapping dataset, with a significantly higher resolution of 3840×2160 pixels, required a longer acquisition time of 30 h and occupies the largest storage space of 68.5 GB. The structured approach ensures that each dataset variant serves distinct use cases, balancing between image detail, coverage redundancy, and processing overhead.

3.2 Automated Dataset Generation

Manual dataset preparation for each mission was time-consuming. To streamline this, an automated system using PyAutoGUI [13] enabled hands-free interaction with Google Earth Pro [1] for consistent, error-free image capture. A Python script managed the process, with the algorithm detailed below.

To systematically capture non-overlapping images for dataset generation, incremental changes in latitude (ΔLat) and longitude ($\Delta Long$) are applied. This ensures uniform coverage of the target area without redundancy. The increments in latitude and longitude are calculated using Eqs. 3 and 4, respectively.

- ΔLat represents the north-south increment between consecutive images. According to Snyder [12], 1° of latitude corresponds to approximately 111.32 km, hence ΔLat is calculated accordingly.

$$\Delta Lat = \frac{\text{Ground Height}}{111,320} \quad (3)$$

- $\Delta Long$ corresponds to the east-west increment and varies with latitude due to Earth's curvature. It is adjusted using the cosine of the latitude (θ):

$$\Delta Long = \frac{\text{Ground Width}}{111,320 \times \cos(\theta)} \quad (4)$$

- **(Lat₁, Long₁) and (Lat₂, Long₂)** define the rectangular area on Google Earth, marking the top-left and bottom-right corners, respectively. These coordinates are used for mapping and dataset generation.

As shown in Algorithm 1, the process automates image capturing within a specified region by incrementally updating latitude and longitude values. It systematically navigates through each coordinate point, captures the image using Google Earth Pro, and saves it with a corresponding row-column naming convention. This ensures uniform and complete coverage of the selected area for dataset creation.

3.3 Data Creation Application

The Data Creation and Stitching (D-CAS) application is a tool designed for generating aerial image datasets for specific geographic areas. By utilizing the Google Earth Pro windows application, D-CAS automates the data creation process, making it efficient and user-friendly. The application works through the following steps:

Algorithm 1. Automated Image Capture Based on Latitude and Longitude Increments.

```

1: Initialize  $row \leftarrow 1, col \leftarrow 1$ 
2: Launch Google Earth Pro and load target area
3: Adjust map settings: hide sidebar, uncheck overlays, set max resolution
4: Initialize coordinates:  $x \leftarrow Lat_1 + \frac{\Delta Lat}{2}, y \leftarrow Long_1 + \frac{\Delta Long}{2}$ 
5: while  $x \leq Lat_2$  do
6:   while  $y \leq Long_2$  do
7:     Capture and save image at coordinates  $(x, y)$  as image name (img
        $row, col.png$ ).
8:     Update longitude:  $y \leftarrow y + \Delta Long$  and  $col \leftarrow col + 1$ 
9:   end while
10:  Update latitude:  $x \leftarrow x + \Delta Lat, row \leftarrow row + 1$  and  $col \leftarrow 1$ 
11: end while
12: End process after all images are captured

```

1. **Upload KML File:** The user uploads a KML file that defines the boundaries of the area of interest.
2. **Configure Options:** The user selects two key settings:
 - **Overlapping:** Specify whether overlapping between images is required.
 - **Image Quality:** Set the desired resolution for aerial images.

As illustrated in Fig. 1, the interface clearly demonstrates the steps involved in uploading the KML file and configuring the options within the DCAS application.

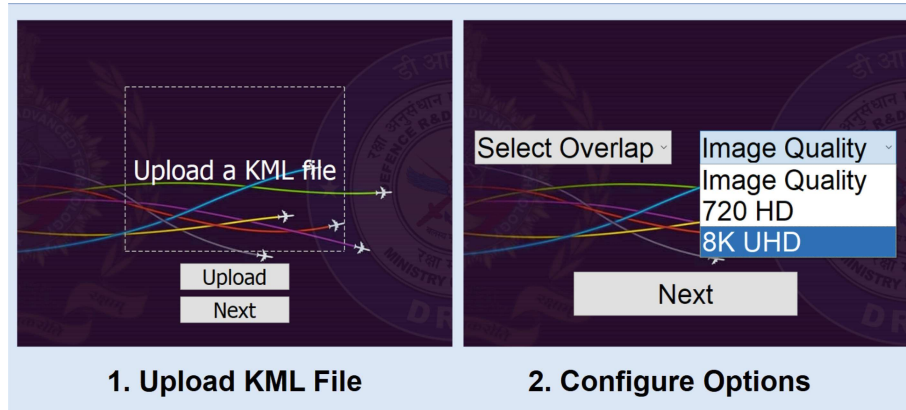


Fig. 1. Uploading a KML file and Configure Options in DCAS App.

3. **Generate Dataset:** Click the 'Start Generation Dataset' button to begin. The application captures aerial images based on the configured settings and saves them in a folder using a matrix-style naming convention format:

- Row 1: $img1,1, img1,2, \dots, img1,n$
- Row 2: $img2,1, img2,2, \dots$, and so on.

As shown in Fig. 2, the dataset contains a variety of aerial images captured using the DCAS application, including features such as roads, rivers, farmland, industrial areas, and other land types.



Fig. 2. Sample Images From Dataset

3.4 Step-by-Step Workflow of the Proposed DSMAC Scheme

Step 1: Dataset Generation

- **Input:** User uploads a KML file with defined area boundaries.
- **Parameters Set:** Image quality, overlapping ratio, and altitude are configured.
- **Grid Generation:**
 - Calculate ΔLat and $\Delta Long$ based on altitude and the field of view.
 - Initialize coordinates for a rectangular grid from $(Lat_1, Long_1)$ to $(Lat_2, Long_2)$.
- **Automated Capture:**
 - Use PyAutoGUI in combination with Google Earth Pro to automate image capturing.
 - Capture images systematically in a row-wise and column-wise manner.
- **Save Images:**
 - Store images using a matrix-style naming convention (e.g., $img1,1$, $img1,2$, etc.).

Step 2: Model Training and Evaluation

- **Traditional Approach (Baseline):**

- Use ORB (Oriented FAST and Rotated BRIEF) for keypoint detection and descriptor extraction.
- **Deep Learning Approach:**
 - Extract features using a ResNet-50 model with the final fully connected layer removed.
 - Generate 2048-dimensional feature vectors and store them in a FAISS index.
- **Matching Process:**
 - For each test image:
 - * Extract features using the same ResNet-50 backbone.
 - * Use FAISS to retrieve the top- k most similar images.
- **Comparison and Evaluation:**
 - Compare accuracy, speed, and robustness of ORB and FAISS models.
 - Evaluate under various conditions such as changes in altitude, image tilt, and overlapping vs. non-overlapping datasets.

Step 3: Deployment and Real-Time Use

- **Input:** A real-time aerial image is captured (E.g. from a Autonomous Aerial Navigation Systems).
- **Process:**
 - Extract the feature vector using ResNet-50.
 - Query the FAISS index using the extracted features.
 - Retrieve the top matching dataset images.
- **Output:**
 - The coordinates of the matched image are used to inform navigation decisions.

3.5 Model Evaluation

After constructing a robust dataset, the next step was to evaluate various models to determine their performance and suitability for the task. The evaluation began with traditional feature-based methods, followed by exploring advanced deep learning architectures and scalable indexing techniques.

Initially, the ORB (Oriented FAST and Rotated BRIEF) model was employed as a baseline for image matching, leveraging its simplicity and efficiency. Building upon this, a U-Net model was implemented to exploit its powerful segmentation capabilities, aiming to enhance accuracy in feature extraction and matching. Finally, FAISS (Facebook AI Similarity Search) was utilized in conjunction with ResNet-50, a pre-trained deep learning model, to enable fast and scalable image similarity searches.

The following subsections provide a detailed analysis of the implementation and performance of each model.

Oriented FAST and Rotated BRIEF (ORB). ORB combines two key techniques for efficient feature detection: the FAST algorithm for keypoint identification and the BRIEF descriptor for feature representation.

FAST (Features from Accelerated Segment Test), introduced by Rosten and Drummond [9], identifies a pixel p as a keypoint if its intensity significantly differs from those in a surrounding circle. This process is defined in Eq. 5:

$$\text{FAST}(p) = \text{True if } p \text{ is a keypoint.} \quad (5)$$

BRIEF (Binary Robust Independent Elementary Features), proposed by Calonder et al. [3], generates a binary descriptor by comparing pixel intensities within a patch around the detected keypoint p , as shown in Eq. 6:

$$\text{BRIEF}(p) = [\text{Compare}(p_i, p_j)]_{i,j \in P_p}. \quad (6)$$

Since BRIEF lacks rotation invariance, Rublee et al. [10] addressed this in the ORB method by estimating the orientation $\theta(p)$ of each keypoint using intensity-weighted moments. This angle is used to rotate the descriptor to achieve rotation invariance, as described in Eq. 7:

$$\theta(p) = \text{atan2} \left(\sum_i y_i, \sum_i x_i \right). \quad (7)$$

To support spatial organization, especially for aerial datasets, images are arranged on a 2D grid. The number of grid columns and rows is determined using Eq. 8, while coordinates of each image are computed using Eq. 9:

$$\text{grid_size_x} = \lfloor \sqrt{N} \rfloor, \quad \text{grid_size_y} = \left\lceil \frac{N}{\text{grid_size_x}} \right\rceil, \quad (8)$$

$$r = \left\lfloor \frac{i}{\text{grid_size_y}} \right\rfloor, \quad c = i \bmod \text{grid_size_y}. \quad (9)$$

Finally, ORB features-including keypoints and their binary descriptors-are extracted from an image I as:

$$\text{ORB Features}(I) = \{\text{keypoints}, \text{descriptors}\}. \quad (10)$$

FAISS Fine-Tuning for Image Matching. FAISS (Facebook AI Similarity Search), introduced by Johnson et al. [6], is a high-performance library designed for similarity search and clustering in large datasets. In this work, FAISS is adapted for fast and scalable aerial image matching.

Modifications and Workflow:

- **Feature Extraction Using ResNet-50:** ResNet-50, developed by He et al. [4], is used without its final fully connected layer to extract 2048-dimensional feature vectors from aerial images. The process is defined in Eq. 11:

$$F(x) = \text{Flatten}(\text{ResNet}_{50}(x)). \quad (11)$$

- **Index Creation:** Feature vectors are indexed using FAISS after applying L2-normalization. The distance between two vectors x and y is calculated using the Euclidean distance as shown in Eq. 12:

$$d(x, y) = \|x - y\|_2^2. \quad (12)$$

- **Optimizations:**
 - *Batch Processing:* Indexing is performed in batches to improve computational efficiency.
 - *Dimensionality Reduction:* PCA is optionally applied to reduce the dimensionality of feature vectors, speeding up retrieval without major accuracy loss.

FAISS Workflow Summary:

1. **Feature Extraction:** Generate 2048-dimensional feature vectors from images using ResNet-50 (Eq. 11).
2. **Indexing:** Add feature vectors into a FAISS index with optional PCA-based compression.
3. **Image Matching:** Retrieve the top- k most similar images using L2 distance (Eq. 12).

4 Results and Discussion

4.1 Results

ORB MODEL Results: The ORB model demonstrated high accuracy (86–92%) across all dataset types but required significantly more processing time (244–523 seconds for testing). **Figure 3** illustrates three key test scenarios where ORB successfully matched images under varying conditions:

1. **Direct Image Matching:** ORB accurately matched a test image with the correct dataset image, demonstrating basic matching capability.
2. **Altitude Adjustment Matching:** ORB successfully matched a high-altitude image scaled to 100 m, handling scale variations effectively.
3. **Tilted Image Matching:** ORB correctly matched a tilted 50-meter altitude image, proving robust against scale and orientation changes.

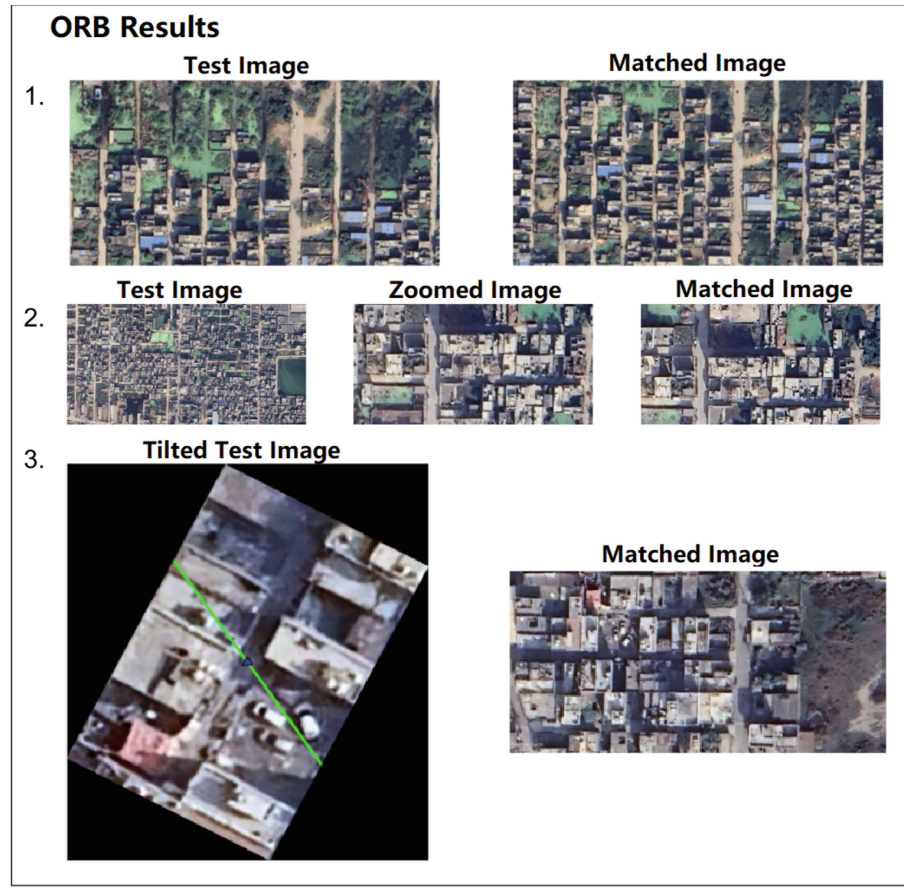


Fig. 3. Oriented Fast and Rotated Brief (ORB) Results

Table 3. Performance comparison of ORB model

ORB	Training Time (sec)		Testing Time (sec)		Accuracy
	CPU	T4 GPU	CPU	T4 GPU	
Low Q^* with O^*	4207	3682	523	523	86%
Low Q^* without O^*	408	342	244	244	86%
High Q^* with O^*	5128	*	498	*	92%

* These numbers could not be calculated due to resource constraints.

* Q means Quality, *O means Overlapping

Table 3 shows ORB's strong performance, with 86% accuracy on low-quality data and 92% on high-quality overlapping images. While accuracy remains high, training time increases with data size, reaching 5128 s on CPU.

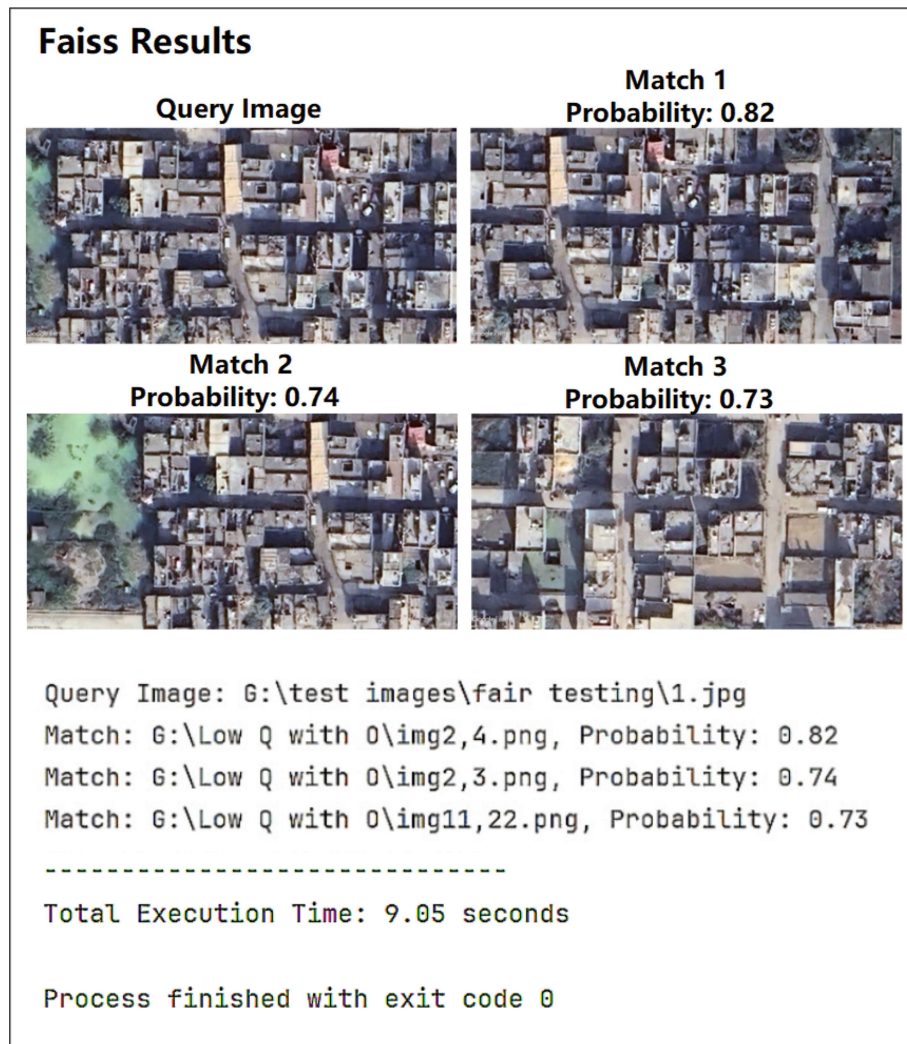


Fig. 4. Facebook AI Similarity Search (FAISS) Results.

FAISS MODEL Results: FAISS processed 50 aerial images in under 10s, achieving **5 images per second**. For each query, it provided the top three nearest matches with probability scores (Fig. 4).

Figure 4 illustrates FAISS’s ability to retrieve the top three matching aerial images along with probability, showcasing its speed and accuracy on large datasets.

Compared to **ORB**, **FAISS** is faster, more robust, and highly efficient, making it ideal for large-scale image retrieval.

As seen in Table 4, the FAISS model using ResNet-50 achieves the highest accuracy (88%) when both high image quality and overlapping are used. However, this also results in significantly higher training times, especially on CPU. Lower quality or the absence of overlapping reduces both accuracy and training time. Values marked with * were not computed due to hardware limitations.

Table 4. Performance comparison of FAISS (Res-net 50)

FAISS (Res-net 50)	Training Time (sec)		Testing Time (sec)		Accuracy
	CPU	T4 GPU	CPU	T4 GPU	
Low Q * with O *	2201	3606	12	10	86%
Low Q * without O *	1275	897	14	9	80%
High Q * with O *	6219	*	16	*	88%

* These numbers could not be calculated due to resource constraints.

* Q means Quality

* O means Overlapping

4.2 Key Observations

- The ORB model is computationally efficient but struggles with feature-sparse images.
- FAISS offers a balance of speed and accuracy, particularly excelling in scenarios where feature-rich images are available.
- These results underline the importance of selecting models based on the specific characteristics of the dataset and the application requirements.

This analysis highlights the trade-offs between computational efficiency, testing time, and accuracy, providing insights for selecting the most suitable model for aerial image matching tasks.

As shown in Table 5, ORB offers high accuracy but is slower, making it less ideal for real-time use. FAISS, in contrast, is much faster and scalable, though it relies on pre-trained embeddings for effective performance.

As seen in Table 6, our FAISS-based system achieves competitive accuracy (88%) with the fastest processing speed (under 1s), making it highly suitable for real-time aerial navigation. Other state-of-the-art methods offer comparable or slightly higher accuracy but with slower performance.

Our proposed system shows competitive performance compared to state-of-the-art methods, with superior processing speed that makes it particularly suitable for real-time navigation applications. While Wang et al. [14] achieve marginally higher accuracy, our system processes images over three times faster, representing a significant advantage for time-critical aerial navigation scenarios.

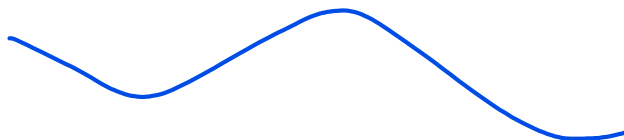


Table 5. Model Comparison Overview

Model	Accuracy	Processing Time	Strengths	Limitations
ORB	High	15 s	Accurate matching of keypoints	Too slow for real-time use
FAISS	Moderate	less than 1 s	Efficient, robust and scalable	Required pre-trained embeddings

^a The processing time is evaluated under specific hardware configurations. Pre-trained embeddings refer to models trained on large datasets to improve performance in tasks like feature matching

Table 6. Comparison with State-of-the-Art Image Matching Systems

System	Accuracy	Processing Speed	Key Advantage
Our FAISS-based System	88%	Less than 1 s	Optimized for aerial navigation
Zhou et al. [15]	83%	2.3 s	Fusion with inertial data
Nassar et al. [7]	86%	1.5 s	Enhanced UAV geolocation
Wang et al. [14]	91%	3.2 s	Higher precision but slower

4.3 Discussion

The comparative analysis of the implemented models reveals several important insights for aerial image matching applications:

- The ORB model demonstrated slower computation but higher accuracy on feature-sparse images, while FAISS (ResNet-50) excelled in speed for feature-rich datasets but delivered moderate accuracy.
- Hardware acceleration using a T4 GPU reduced the training and testing times, highlighting its importance for deep learning tasks. The most significant improvements were observed in the FAISS model, where GPU processing reduced testing time by approximately 35%.
- A trade-off was observed between computational efficiency and accuracy, with ORB being slower but more accurate, while FAISS provided faster results with moderate accuracy for complex datasets. This trade-off is particularly evident in navigation scenarios where real-time processing is critical.
- The overlapping images dataset consistently yielded better results across both models, indicating that feature redundancy enhances matching reliability, especially in environments with sparse distinguishing features.

These findings suggest that a hybrid approach, potentially combining the strengths of traditional feature-based methods with deep learning techniques, might be optimal for mission-critical aerial navigation systems. Such an approach could adapt to different environmental conditions and computational constraints.

5 Challenges and Limitations

The development and testing of the proposed image matching system faced several challenges:

- **Daytime Navigation Only:** The current model is optimized for daylight conditions and cannot perform effectively under low-light or nighttime scenarios, limiting its applicability in 24-hour operations. This is primarily due to the training dataset being collected during daylight hours, resulting in features that are not invariant to significant lighting changes.
- **Terrain Limitations:** The model was trained primarily on urban environments, making it unsuitable for terrains like seas, dense forests, or mountainous regions, where image features are less distinct. The feature extraction methods employed struggle with homogeneous textures that lack clear edges or corners for keypoint detection.
- **High Computational Requirements:** Real-time performance depends on powerful GPUs, which can limit deployment on resource-constrained platforms, such as lightweight drones or compact navigation systems. While FAISS significantly improves processing speed, the initial feature extraction using deep learning models remains computationally intensive.
- **Dataset Generalization:** The current dataset, though extensive, represents specific geographical locations. The model's performance might degrade when applied to regions with substantially different visual characteristics or seasonal variations not represented in the training data.

These limitations provide valuable direction for future research and development efforts to enhance the robustness and versatility of the system.

6 Conclusion and Future Work

The proposed research has demonstrated the potential of AI-driven image matching systems to address critical challenges in navigation, particularly in GPS-denied environments. Through the development of a CNN-based framework, the system achieved robust and efficient image matching, supported by a custom dataset. Preliminary results validate the adaptability and reliability of the approach under diverse environmental conditions.

Our key contributions include:

- Development of an automated dataset generation workflow that significantly reduces the time and effort required to create mission-specific aerial image datasets.
- Implementation and comparative analysis of traditional and deep learning-based image matching approaches, highlighting their respective strengths and limitations.
- Performance optimization techniques for real-time processing, particularly through the adaptation of FAISS for aerial image matching.

- A flexible framework that can be adapted to various aerial navigation applications, including missile guidance systems operating in GPS-denied environments.

Future Work

- Explore advanced architectures like Vision Transformers for improved performance in feature-sparse environments and varying lighting conditions.
- Optimize FAISS for faster processing without compromising accuracy, potentially through efficient model quantization techniques.
- Focus on real-time aerial image matching for practical applications, including hardware-specific optimizations for embedded systems.
- Augment the dataset to improve generalization across feature-sparse images and diverse environmental conditions, including seasonal variations and different times of day.
- Investigate multi-modal approaches that combine visual data with other sensor inputs for enhanced navigation reliability.
- Develop adaptive models that can dynamically adjust their processing strategies based on the computational resources available and the environmental conditions encountered.

The research presented lays a solid foundation for advancing the capabilities of navigation systems in challenging environments where conventional GPS-based methods are unreliable or unavailable. By continuing to refine these approaches and addressing the identified limitations, future implementations can provide robust, efficient, and adaptable navigation solutions for a wide range of aerial platforms.

Acknowledgment. We are thankful to **Ms. Premila C. M.**, Technology Director, **Mr. Y Dilip**, Outstanding Scientist and the Director, **Aeronautical Development Establishment, DRDO**, Bengaluru for their support and guidance.

References

1. Google earth pro (2025). <https://www.google.com/earth/versions/>. Accessed 23 July 2025
2. Bai, Y.: Research of image detection and matching algorithms. In: International Conference on Information Engineering and Applications (ICIEA) (2023)
3. Calonder, M., Lepetit, V., Strecha, C., Fua, P.: BRIEF: binary robust independent elementary features. In: Daniilidis, K., Maragos, P., Paragios, N. (eds.) ECCV 2010. LNCS, vol. 6314, pp. 778–792. Springer, Heidelberg (2010). https://doi.org/10.1007/978-3-642-15561-1_56
4. He, K., et al.: Deep residual learning for image recognition. In: Computer Vision and Pattern Recognition, pp. 770–778. IEEE (2016)
5. Irani, G., Christ, J.: Image processing for tomahawk scene matching. Johns Hopkins APL Tech. Digest **15**(3) (1994)

6. Johnson, J., et al.: Billion-scale similarity search with GPUs. arXiv preprint [arXiv:1702.08734](https://arxiv.org/abs/1702.08734) (2017)
7. Nassar, M., et al.: A deep CNN-based framework for enhanced aerial imagery registration with applications to UAV geolocalization. *Remote Sens.* **10**(11), 1782 (2018)
8. Ronneberger, O., Fischer, P., Brox, T.: U-net: convolutional networks for biomedical image segmentation. In: Navab, N., Hornegger, J., Wells, W.M., Frangi, A.F. (eds.) *MICCAI 2015*. LNCS, vol. 9351, pp. 234–241. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-24574-4_28
9. Rosten, E., Drummond, T.: Machine learning for high-speed corner detection. In: Leonardis, A., Bischof, H., Pinz, A. (eds.) *ECCV 2006*. LNCS, vol. 3951, pp. 430–443. Springer, Heidelberg (2006). https://doi.org/10.1007/11744023_34
10. Rublee, J., et al.: ORB: an efficient alternative to SIFT or SURF. In: *International Conference on Computer Vision*, pp. 2564–2571. IEEE (2011)
11. Sarlin, P.E., DeTone, D., Malisiewicz, T.: SuperGlue: learning feature matching with graph neural networks. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2020)
12. Snyder, J.: *Map Projections: A Working Manual*, vol. 1395. U.S. Geological Survey, Washington, DC (1987)
13. Sweigart, A.: PyAutoGUI: GUI automation for Python. Python Package Index (PyPI) (2025). Version 0.9.53. <https://pypi.org/project/PyAutoGUI/>. Accessed 23 July 2025
14. Wang, H., et al.: Precise aerial image matching based on deep homography estimation. *J. Intell. Rob. Syst.* **104**(3), 25 (2021)
15. Zhou, L., Zhang, H., Liu, J.: A novel scene matching navigation system for UAVs based on vision/inertial fusion. *IEEE Access* **6**, 55462–55471 (2023)